

INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

Department of Electrical Engineering - ICDT

PROJECT REPORT

Design and Calibration of an Arduino UNO-Based Digital Clock

Submitted by:

Rudra Vishweshwar (EE26BTECH11225)

Saksham Sawant (EE26BTECH11226)

Submitted To:

Prof Dr.GVV Sharma

Teaching Assistants:

Vivek Kumar, Subhodeep Chakraborty , Jnanesh

Date of Submission:

August 28, 2026

Table of Contents

1 Introduction 1

2 Project Objectives 1

3 Physical Design & Components 2

3.1 Subsystem Breakdown . . . 2

3.2 List of Materials . . . 2

3.3 Importance of Decoder, Resistors for Displays . . . 3

3.4 The 4-Pin Data Lines . . . 3

3.5 The Multiplexing Control Lines . . . 4

3.6 The Interactive Control Pad . . . 4

3.7 Our Arduino Wire Mapping . . . 5

4 How the Code Counts Time 6

4.1 Our Shifting Logic Switches . . . 6

4.2 The Code Layout Rules . . . 6

4.3 How We Figured Out the Logic Using K-Maps . . . 6

4.4 Shorter Logic for the Tens Columns . . . 7

4.5 Our Clock Code . . . 8

5 Finding and Fixing Physical Bugs 9

5.1 Removing of the External Resistors . . . 9

5.2 Managing the Breadboard Wire Chaos . . . 10

5.3 Our Troubleshooting Summary . . . 10

6 Our Project Development Diary 11

6.1 Phase 1: Facing the Hardware Reality . . . 11

6.2 Phase 2: The Boring and Tedious Code Breakdown . . . 11

6.3 Phase 3: The Resistor Failure and the Software Pivot 12

7 Project Summary and Conclusions 13

List of Tables

1 List of Materials 2

2 BCD Bit Weights and Binary Logic States for Decimal Digits (0–9) 4

3 Arduino Hardware Pin Assignment 5

4 Our Paper K-map for the MSB (Z_{next}) Switch 7

5 Bench Troubleshooting Log 10

1 Introduction

This project focuses on gaining hands-on engineering experience by building a physical digital clock from the component level on a breadboard.

Instead of using highly integrated modules, the system physically interfaces an Arduino UNO microcontroller with a discrete **7447 BCD-to-Seven Segment Decoder IC**.

This practical approach shifts the focus away from software simulation and moves directly into physical circuit tracing, manual wire management, and discrete component handling.

Working with real hardware parts establishes a foundational understanding of troubleshooting while highlighting the importance of neat wire organization and real-time electronic diagnostics.

2 Project Objectives

The primary objective of this project is to construct a hardware-centric digital clock from the component level using an Arduino UNO. The specific technical goals are:

- **Hardware Interfacing:** Physical mapping of Arduino GPIO pins to a discrete 7447 BCD-to-Seven Segment Decoder IC.
- **Discrete Logic Control:** Using basic active-low decoding instead of pre-built display modules.
- **Timing Implementation:** Designing a reliable firmware clock engine to accurately update seconds, minutes, and hours.
- **Clock Calibration:** Correcting the microcontroller's timing drift to ensure real-world clock accuracy.
- **Hardware Diagnostics:** Developing systematic breadboard wire management and manual troubleshooting skills.

3 Physical Design & Components

The digital clock is implemented entirely using discrete hardware components distributed for the laboratory coursework. This section details the complete list of materials used in the physical assembly.

3.1 Subsystem Breakdown

The overall structure splits the received components into three clear hardware layers:

- **Control Layer:** The Arduino UNO uses code loops to track hours, minutes, and seconds, generating a 4-bit parallel binary representation of each digit.
- **Decoding Layer:** The 7447 IC intercepts the 4-bit binary values and maps them directly to active-low logical ground signals.
- **Display Layer:** The common anode seven-segment display routes 5V internally to all LEDs, illuminating numbers whenever the 7447 chip pulls a specific segment pin to ground.

3.2 List of Materials

Table 1 outlines the specific parts received and integrated into the breadboard setup.

Table 1: List of Materials

S.No.	Component Name	Quantity	Specification / Value
1	Arduino UNO Board	1	ATmega328P Microcontroller
2	7447 BCD Decoder IC	1	16-pin Active-Low Driver
3	7-Segment Display	6	Common Anode (Red)
4	Resistors	11	7 (220 Ω) + 4 (10$K\,\Omega$)
5	Breadboard	1	Standard Full-Size Rail
6	Jumper Wires	90	Male-to-Male (Solid Core)
7	USB A-to-B Interface Cable	1	Power & Serial Programming Line
8	Tactile Buttons	4	(2)2 pinned + (2)4 pinned

3.3 Importance of Decoder, Resistors for Displays

Each decimal digit on the main display is processed as four BCD bits, where A is the 1's bit, B is the 2's bit, C is the 4's bit, and D is the 8's bit. For example, decimal 5 is represented on our parallel bus as binary `0101`.

The (SN74LS47N)IC decoder chip uses these bits to select the correct active-low segment patterns natively. To protect the common-anode LED segments from terminal over-current damage, seven $240\,\Omega$ resistors are mapped along the segment paths based on Ohm's law:

$$R \approx \frac{V_{\text{supply}} - V_F}{I} \tag{1}$$

Where V_{supply} is 5V, V_F is the forward voltage drop of an illuminated segment, and I is our desired safe operational current.

- To prevent our prototype breadboard from turning into a total mess of loose connections,we organized our physical wires into three simple code-defined groups. By running four parallel data lines into the hardware inputs of the SN74LS47N chip and multiplexing our digits sequentially, we significantly dropped our total pin connections.

3.4 The 4-Pin Data Lines

The firmware condenses our clock variables into a 4-bit parallel stream that runs straight to the decoding chip. These pins map directly to the binary weights of our numbers:

- **`PIN_A = 4:`** Controls Binary Bit 0 (The Least Significant Bit).
- **`PIN_B = 5:`** Controls Binary Bit 1.
- **`PIN_C = 6:`** Controls Binary Bit 2.
- **`PIN_D = 7:`** Controls Binary Bit 3 (The Most Significant Bit).

Table 2: BCD Bit Weights and Binary Logic States for Decimal Digits (0–9)

Decimal Digit	**Bit D (2^3 / MSB)**	**Bit C (2^2)**	**Bit B (2^1)**	**Bit A (2^0 / LSB)**
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

3.5 The Multiplexing Control Lines

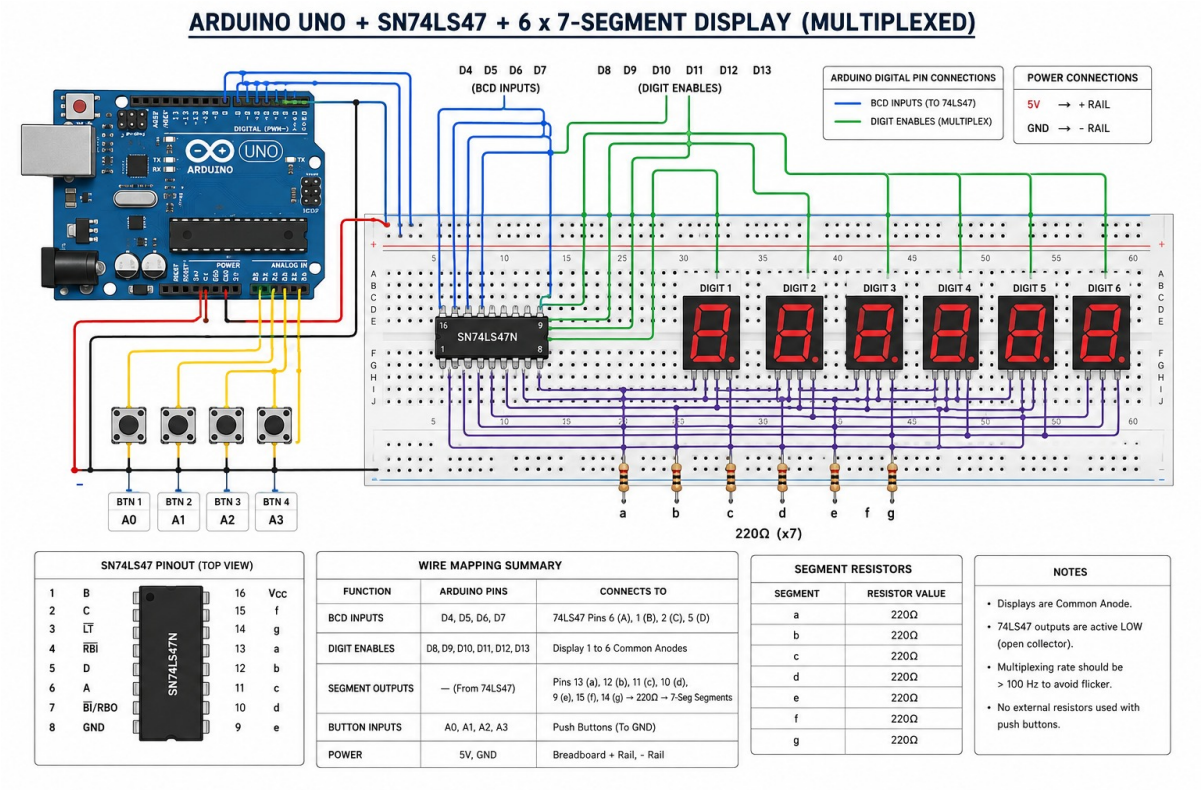
The code defines an array of active-high enable lines to sequence through our digits fast enough to trick the human eye. These digital outputs map directly to our panel array:

- **`EN[] = {13, 12, 11, 10, 9, 8}`:** Handles sequential digit selection from the first seconds panel down to the tens-of-hours panel.

3.6 The Interactive Control Pad

To avoid using messy physical hardware components on our board, the control buttons are run directly into the analog pins using internal microcontroller pull-ups:

- **`PAUSE_BTN = A0`:** Freezes time tracking and takes the system into calibration mode.
- **`NEXT_BTN = A1`:** Helps as a cursor to select which digit of clock is to be updated.
- **`INC_BTN = A2`:** Increments the value of the selected digit.
- **`DEC_BTN = A3`:** Decrements the value of the selected digit.



3.7 Our Arduino Wire Mapping

Table 3 documents the exact physical pin connections running across our breadboard rails, routing display signals directly from Segment A down through Pin 2.

Table 3: Arduino Hardware Pin Assignment

Arduino Pins	**Connected Component**	**Functional Hardware Role**
Pins 2, 3, 4, 5, 6, 7, 8	SN74LS47N Decoder Outputs	Maps directly to segments a, b, c, d, e, f, g on the displays.
Pins 4, 5, 6, 7	SN74LS47N Binary Inputs	Feeds our 4-bit synchronous logic values to the translator chip.
Pins 8, 9, 10, 11, 12,13 (Multiplexed)	Display Common Anodes	Handles high-speed panel cycling to flash individual digit frames.
Analog Input Pins A0-A3	Tactile Push Buttons	Controls the calibration menu (Pause, Next, Increment, Decrement).

4 How the Code Counts Time

Our program does not use normal addition to track time. Instead, it moves through binary states using a custom set of software switches.

4.1 Our Shifting Logic Switches

Every digit of numbers use four independent true/false switches in the code: `W`, `X`, `Y`, and `Z`. Together, these four variables form the binary code for the digit showing on the screen.

4.2 The Code Layout Rules

When the clock ticks or a button is pressed, the code shifts the states based on four crisp rules:

- Switch `W` simply flips to its opposite state on every single tick.
- Switch `X` only flips when specific even-number boundaries are crossed.
- Switch `Y` checks switches `W` and `X` to know when to shift.
- Switch `Z` only turns on when `W`, `X`, and `Y` are active at the same time.

The program clears these switches back to zero only when a column hits its specific limit (like resetting the tens-of-minutes column back to zero after counting up to 5).

4.3 How We Figured Out the Logic Using K-Maps

Honestly, we didn't just guess those four switching rules; we struggled to understand it. Trying to track how four binary switches (W, X, Y, Z) should flip between numbers 0 and 9 got messy quickly. To prevent our code from turning into a nightmare of endless "if-else" loop statements, we are going to use Karnaugh Maps (K-maps) to simplify the code.

Our favorite shortcut was using "don't-care" states. Since our clock only counts from 0 to 9, numbers 10 to 15 are completely impossible. We

treated those unused slots as wildcards (×) on our planning grid. This allowed us to group our numbers into much bigger blocks, simplifying our math down to just a few clean lines of code:

$$\begin{aligned}
\text{Equation 1 (Switch W):} \quad & W_{next} = !W \\
\text{Equation 2 (Switch X):} \quad & X_{next} = X \,\hat{}\, (W \;\&\; !Z) \\
\text{Equation 3 (Switch Y):} \quad & Y_{next} = Y \,\hat{}\, (W \;\&\; X) \\
\text{Equation 4 (Switch Z):} \quad & Z_{next} = (W \;\&\; X \;\&\; Y) \mid (Z \;\&\; !W)
\end{aligned}$$

To make sense of these equations, we mapped them directly to standard programming logic symbols rather than confusing textbook math notations:

- **!** (NOT) — Flips a switch to its exact opposite state.
- **&** (AND) — Requires both conditions to be true at the same time.
- **^** (XOR) — Acts as a toggle switch (flips state only when triggered).
- **|** (OR) — Triggers if either condition is met.

To visualize how this actually worked on paper, Table 4 shows our K-map layout for the heaviest switch (Z_{next}). You can clearly see where we grouped the active signals and ignored the impossible wildcard states to save processing power.

Table 4: Our Paper K-map for the MSB (Z_{next}) Switch

WX \ YZ	**00**	**01**	**11**	**10**
00	0	0	×	×
01	0	0	×	×
11	0	1	×	×
10	0	1	×	×

4.4 Shorter Logic for the Tens Columns

We had to tweak this logic for the tens-of-minutes and tens-of-seconds columns. Those numbers only need to count from 0 to 5 before resetting to zero. This meant states 6 through 15 became "don't-care" states. Truncating the map this early allowed us to cut out even more code, making the multiplexing cycles run incredibly fast and crisp on the actual breadboard.

4.5 Our Clock Code

Source Code Link:
Click Here for CODE

Opening this link provides direct access to the complete Arduino source code document for verification and deployment.

5 Finding and Fixing Physical Bugs

Building a real circuit on a plastic breadboard causes practical glitches that a clean computer simulation never shows. We isolated these anomalies and corrected them directly using firmware updates on our desk.

5.1 Removing of the External Resistors

Our first attempt at wiring the buttons used physical resistors plugged into the breadboard as pull-downs.

[A pull-down resistor connects an unpressed button pin directly to Ground (0V) to prevent it from picking up electrical noise (this situation is known as FLOATING PINS), which ensures that the line stays steadily at LOW until you press the switch to force it HIGH (5V).]

Because breadboard holes can be loose (or atleast that's what we thought), the electrical signals bounced around randomly. This caused the clock to skip values, register fake clicks, or get completely stuck in the menu.

Out of doubt and frustration, we pulled those resistors out of the board entirely, which actually worked better. So I fixed it in software by changing our inputs to use internal settings:

```
pinMode(PAUSE_BTN, INPUT_PULLUP);
pinMode(NEXT_BTN,  INPUT_PULLUP);
pinMode(INC_BTN,   INPUT_PULLUP);
pinMode(DEC_BTN,   INPUT_PULLUP);
```

This tells the Arduino to use tiny resistors that are already built inside its own computer chip. It keeps the signal line perfectly steady until you press a button to connect the line to ground. This cleared up our wire mess and made our buttons 100% reliable without a single physical resistor on the board.

5.2 Managing the Breadboard Wire Chaos

Outside of our software fixes, our biggest physical headache was managing the raw density of nearly 90**-irritably fragile-**jumper wires jammed onto a single board. Because the seven 220 Ω current-limiting resistors had long, exposed metal legs, they easily bent out of place under the weight of the messy wiring nest.

Also we should keep in mind that it was due to MULTIPLEXING that we only got to work with approx-90 wires. We can't imagine the level of difficulty of working without it as it would bring in way many more wires and problems.

During several test runs, adjacent resistor legs accidentally came into direct physical contact. These crossings created systemic short-circuits that caused random display segments to permanently lock up or flash erratically. We solved this tedious issue by systematically combing through the layout, trimming down the exposed resistor leads with pliers, and creating manual spacing between them.

5.3 Our Troubleshooting Summary

Table 5 summarizes the exact errors we faced on our desk and how we systematically resolved them.

Table 5: Bench Troubleshooting Log

Observed Issue	**Root Hardware Cause**	**Our Practical Solution**
Skipped values / menu freezing	Loose external resistor rails on breadboard	Shifted to internal microcontroller `INPUT_PULLUP` switches.
Erratic segment locking	Resistor legs bending into physical contact	Trimmed exposed metal leads down using lab pliers.

6 Our Project Development Diary

This log details the actual, unedited development cycle of our clock build from the first lab session to final submission. It tracks what we messed up, what we found incredibly tedious, and how we actually got the system running.

6.1 Phase 1: Facing the Hardware Reality

Our project started with a massive reality check. At first, we thought sir would ask for a clock using C-Language. Afterwards, we entered the lab hoping to build a standard digital clock using an LCD screen and an automated timekeeping chip. Instead, we were handed a bundle of loose components: an Arduino Uno, a single SN74LS47N decoder chip, a set of common-cathode 7-segment displays, and just 15-jumper wires.

We took our loose components back to our room and spread them out, right on the bed and started mapping the pins. It wasn't a dedicated, high-tech lab bench environment; it was just us sorting through a messy bundle of colored jumper wires and trying to see how the Arduino Uno and the IC chip would work and talk to each other. Without a hardware timekeeping chip, our clock would lose its time the exact second it was unplugged from the laptop.

My partner handled most of the circuit placement [**As** I unknowingly broke few wires and a bit of the arduino case,which made both of us afraid **.**] following hints from instructions from the TAs and classmates, until the components were wired onto the board.

We also had to make multiple trips to the lab for things like : replacing the faulty equipment we received, replacing the given cathode displays with anode, for extra wires and replacing the females wires.

6.2 Phase 2: The Boring and Tedious Code Breakdown

While my partner managed the physical wiring, my job was to take care of the project documentation, firmware and figure out how it actually

functioned. Honestly, reviewing the codebase was the most tedious part of the entire project. Instead of using a simple, readable number counter, the firmware simulates a physical digital logic circuit using pure Boolean math switches (`W, X, Y, Z` variables).

Tracing how these hardcoded logic gates clocked the numbers from 0 to 9 using binary expressions was incredibly exhausting. However, watching the display actually cycle cleanly through seconds for the first time was an immense relief.

6.3 Phase 3: The Resistor Failure and the Software Pivot

Our first major mistake happened when we tried to wire up the four tactile buttons to manually set the hours and minutes. We spent a frustrating amount of time trying to get them to register clicks using the high-value physical resistors from our lab kit as external pull-downs. We don't know what happened but, it caused the clock to skip hours or ignore button presses entirely. We simply could not get the hardware matching right.

Out of sheer frustration, we decided to simplify the circuit and in the thought of rewiring and troubleshooting, we tried without resistors on the breadboard entirely. I updated the initialization script to use the Arduino's internal resistors, setting them to `INPUT_PULLUP`. This single change was the best thing we did. It activated internal pull-up resistors inside the microcontroller chip itself, instantly clearing up our breadboard and making the button presses 100% stable without a single physical resistor.

7 Project Summary and Conclusions

Our project successfully ends with a fully operational, 24-hour digital clock built entirely from scratch on a breadboard. We've managed to build a functional timekeeping engine running, without relying on pre-built modules or automated software libraries.

Ultimately, this project taught us how fragile real hardware connections are compared to perfect software simulations. We are proud that our clock keeps time and responds to button calibration flawlessly.But as it was built under classroom conditions, it has specific real-world limitations that make it different from a finished commercial product:

- **Total Power Dependency:** The system has no onboard battery backup. It must remain plugged into a laptop USB port to function. If the power cable slips out even for a fraction of a second, the entire clock resets back to zero.
- **Loose/Delicate Breadboard Connections:** Because the prototype relies on loose jumper wires plugged into a plastic breadboard, bumping the board too hard can disconnect a digit enable line or disrupt the high-speed multiplexing stream.
- **Manual Calibration:** Correcting for small daily timing drifts relies completely on the accuracy of the internal microcontroller timer, requiring manual adjustments directly inside the firmware code loop.
- **Appearance:** We made this clock to on a breadboard, surrounded by a clumsy mess of jumper wires, which is not good even for it's aesthetics and for its maintenance.

Despite these real-world constraints, taking this project from a confusing bundle of loose lab parts on a bed to a working clock gave us an honest, hands-on understanding of how hardware and software actually interact.